МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
Факультет інформаційно-комп'ютерних технологій
Кафедра інженерії програмного забезпечення

ЗВІТ

з лабораторної роботи № 2
з дисципліни «Розробка мобільних додатків»
на тему:
«Віртуалізація списків та побудова навігації»

Виконала: студентка групи ІПЗ-24-2
Дзядзін Марина Василівна
Перевірив:
Нерода Сергій Іванович

Житомир - 2026

Мета: сформувати практичні навички застосування інструментів віртуалізації списків для оптимізації рендерингу даних, а також проєктування ієрархічних систем навігації та захищених маршрутів у середовищі Expo Router.

ЗАВДАННЯ

1. Створити проєкт Expo, прибрати демонстраційний код командою `npm run reset-project`.
2. Створити контекст авторизації зі станом `isAuthenticated` та функціями `login()` і `logout()`, обгорнути ним кореневий `app/_layout.jsx`.
3. Реалізувати публічний екран входу `app/login.jsx` з автоматичним перенаправленням у застосунок після успішного входу.
4. Створити групу маршрутів `app/(drawer)` із захищеною Drawer-навігацією: неавторизованого користувача перенаправляти на екран входу, для авторизованого показувати `<Drawer>` з кнопкою «Вийти».
5. Створити вкладений Stack `app/(drawer)/(news)` з вимкненим заголовком стека.
6. Реалізувати стрічку новин на `FlatList` (100+ елементів) з Pull-to-Refresh, Infinite Scroll, `getItemLayout` та карткою в `React.memo`.
7. Створити екран деталей `app/(drawer)/(news)/[id].jsx` з отриманням параметра через `useLocalSearchParams`.
8. Створити довідковий центр `app/(drawer)/faq.jsx` на основі `SectionList`.
9. Розмістити код у репозиторії «MobileLabsRN2026» у папці lab2 з README.md (опис, запуск, скріншоти, відповіді на контрольні запитання).

ХІД РОБОТИ

1. Створення проєкту

Проєкт створено на шаблоні Expo SDK 57 (версія, яку підтримує встановлений на смартфоні Expo Go), демонстраційний код прибрано, додаткові бібліотеки встановлено через `npx expo install`. Drawer-навігація в Expo Router використовує бібліотеки `react-native-gesture-handler` та `react-native-reanimated`, які вже входять до шаблону.

Лістинг 2.1 - Створення проєкту

```
npx create-expo-app@latest lab2 --template default@sdk-57
cd lab2
npm run reset-project
npx expo install @expo/vector-icons @expo-google-fonts/montserrat
```

2. Структура маршрутів

Навігація побудована ієрархічно: кореневий Stack містить публічний екран `login` і захищену групу `(drawer)`; усередині Drawer є група `(news)` з власним Stack та екран `faq`. Папки в дужках є групами маршрутів і не потрапляють в URL, тому стрічка новин відкривається за адресою `/`, деталі - за `/5`, довідковий центр - за `/faq`.

Лістинг 2.2 - Структура проєкту lab2

```
lab2/
├── app/
│   ├── _layout.jsx                # AuthProvider + кореневий Stack
│   ├── login.jsx                  # публічний екран входу
│   └── (drawer)/
│       ├── _layout.jsx            # перевірка авторизації + Drawer
│       ├── faq.jsx                # довідковий центр (SectionList)
│       └── (news)/
│           ├── _layout.jsx        # вкладений Stack (headerShown: false)
│           ├── index.jsx          # стрічка новин (FlatList)
│           └── [id].jsx           # деталі новини (динамічний маршрут)
├── components/
│   ├── NewsCard.jsx               # картка новини (React.memo)
│   └── BrandMark.jsx
├── context/AuthContext.jsx        # глобальний стан авторизації
├── data/news.js                   # генератор новин
├── data/faq.js                    # згруповані дані для SectionList
└── constants/theme.js
```

3. Глобальний стан авторизації

Контекст `AuthContext` зберігає поточного користувача та надає похідне значення `isAuthenticated`, а також функції `login()` і `logout()`. Функція `login()` перевіряє формат пошти та довжину пароля (не менше 6 символів) і повертає результат з текстом помилки. Значення контексту мемоїзоване через `useMemo`, а функції - через `useCallback`, щоб споживачі не перерендерювалися без потреби. Хук `useAuth()` спрощує доступ до контексту та перевіряє, що його викликано всередині провайдера.

Лістинг 2.3 - context/AuthContext.jsx

```
import { createContext, useCallback, useContext, useMemo, useState } from 'react';

const AuthContext = createContext(null);

// Глобальний стан авторизації: isAuthenticated + login() / logout()
export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  const login = useCallback((email, password) => {
    const cleanEmail = email.trim().toLowerCase();
    if (!/^\S+@\S+\.\S+$/.test(cleanEmail)) {
      return { ok: false, error: 'Введіть коректну адресу електронної пошти' };
    }
    if (password.length < 6) {
      return { ok: false, error: 'Пароль має містити щонайменше 6 символів' };
    }
    setUser({ email: cleanEmail, name: cleanEmail.split('@')[0] });
    return { ok: true };
  }, []);

  const logout = useCallback(() => setUser(null), []);

  const value = useMemo(
    () => ({ user, isAuthenticated: user !== null, login, logout }),
    [user, login, logout]
  );

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth() має викликатися всередині <AuthProvider>');
  return ctx;
}
```

Провайдер підключено в кореневому макеті, тож стан авторизації доступний на всіх екранах. Корінь також обгорнуто в

`GestureHandlerRootView` (потрібен для жестів Drawer) та `SafeAreaProvider`.

Лістинг 2.4 - app/_layout.jsx

```
import {
  Montserrat_400Regular,
  Montserrat_500Medium,
  Montserrat_600SemiBold,
  Montserrat_700Bold,
  useFonts,
} from '@expo-google-fonts/montserrat';
import { Stack } from 'expo-router';
import { StatusBar } from 'expo-status-bar';
import { ActivityIndicator, View } from 'react-native';
import { GestureHandlerRootView } from 'react-native-gesture-handler';
import { SafeAreaProvider } from 'react-native-safe-area-context';
import { colors } from '../constants/theme';
import { AuthProvider } from '../context/AuthContext';

export default function RootLayout() {
  const [fontsLoaded] = useFonts({
    Montserrat_400Regular,
    Montserrat_500Medium,
    Montserrat_600SemiBold,
    Montserrat_700Bold,
  });

  if (!fontsLoaded) {
    return (
      <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
        <ActivityIndicator color={colors.blue} />
      </View>
    );
  }

  return (
    <GestureHandlerRootView style={{ flex: 1 }}>
      <SafeAreaProvider>
        {/* Провайдер авторизації обгортає всю навігацію застосунку */}
        <AuthProvider>
          <StatusBar style="dark" />
          <Stack screenOptions={{ headerShown: false }}>
            <Stack.Screen name="login" />
            <Stack.Screen name="(drawer)" />
          </Stack>
        </AuthProvider>
      </SafeAreaProvider>
    </GestureHandlerRootView>
  );
}
```

4. Екран входу

Екран входу містить поля Email і Пароль (з кнопкою показу пароля) та кнопку «Увійти». Помилки валідації виводяться під формою й зникають під час введення. Якщо `isAuthenticated` стає `true`, екран повертає `<Redirect`

`href="/" />`, і користувач автоматично потрапляє до стрічки новин. Для коректної роботи з клавіатурою використано `KeyboardAvoidingView`, а відступи від системних елементів забезпечує `SafeAreaView`.

Лістинг 2.5 - app/login.jsx (без стилів)

```
import { Ionicons } from '@expo/vector-icons';
import { Redirect } from 'expo-router';
import { useState } from 'react';
import {
  KeyboardAvoidingView,
  Platform,
  Pressable,
  ScrollView,
  StyleSheet,
  Text,
  TextInput,
  View,
} from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import BrandMark from '../components/BrandMark';
import { colors, fonts, radius } from '../constants/theme';
import { useAuth } from '../context/AuthContext';


export default function LoginScreen() {
  const { isAuthenticated, login } = useAuth();
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [showPassword, setShowPassword] = useState(false);
  const [error, setError] = useState('');


  // Після успішного входу стан змінюється - і користувача автоматично
  // перенаправляє в застосунок
  if (isAuthenticated) return <Redirect href="/" />;


  const onSubmit = () => {
    const result = login(email, password);
    if (!result.ok) setError(result.error);
  };


  return (
    <SafeAreaView style={styles.safe}>
      <KeyboardAvoidingView style={{ flex: 1 }} behavior={Platform.OS === 'ios' ?
'padding' : undefined}>
        <ScrollView contentContainerStyle={styles.container}
keyboardShouldPersistTaps="handled">
          <View style={styles.brand}>
            <BrandMark size={72} />
            <Text style={styles.title}>Новини політехніки</Text>
            <Text style={styles.subtitle}>Увійдіть, щоб переглядати стрічку
новин</Text>
          </View>


          <View style={styles.card}>
            <Text style={styles.label}>Email</Text>
            <View style={styles.inputRow}>
              <Ionicons name="mail-outline" size={18} color={colors.textMuted} />
              <TextInput
                value={email}
                onChangeText={(v) => {
                  setEmail(v);
                  setError('');
                }}
                placeholder="student@ztu.edu.ua"
```

```
                placeholderTextColor={colors.placeholder}
                keyboardType="email-address"
                autoCapitalize="none"
                autoComplete="email"
                style={styles.input}
              />
            </View>

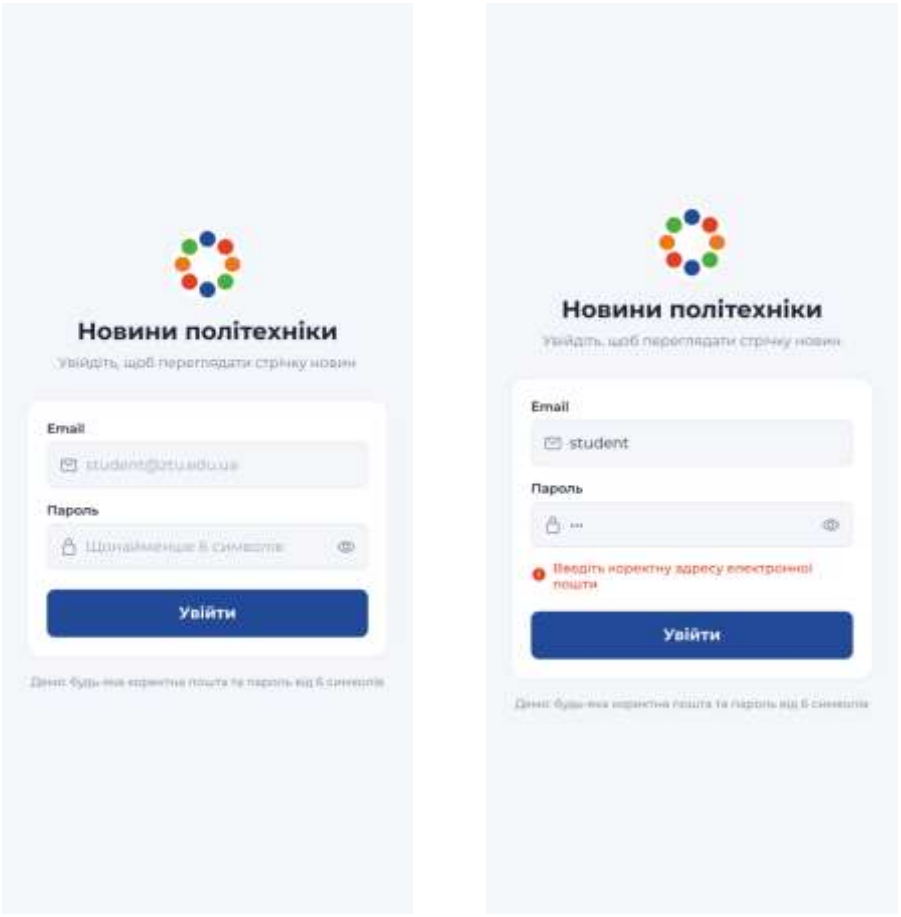
            <Text style={[styles.label, { marginTop: 14 }]}>Пароль</Text>
            <View style={styles.inputRow}>
              <Ionicons name="lock-closed-outline" size={18} color={colors.textMuted}
/>
              <TextInput
                value={password}
                onChangeText={(v) => {
                  setPassword(v);
                  setError('');
                }}
                placeholder="Щонайменше 6 символів"
                placeholderTextColor={colors.placeholder}
                secureTextEntry={!showPassword}
                style={styles.input}
                onSubmitEditing={onSubmit}
              />
              <Pressable onPress={() => setShowPassword((s) => !s)} hitSlop={10}>
                <Ionicons name={showPassword ? 'eye-off-outline' : 'eye-outline'}
size={18} color={colors.textMuted} />
              </Pressable>
            </View>

            {error ? (
              <View style={styles.errorBox}>
                <Ionicons name="alert-circle" size={16} color={colors.red} />
                <Text style={styles.errorText}>{error}</Text>
              </View>
            ) : null}

            <Pressable style={({ pressed }) => [styles.button, pressed && { opacity:
0.85 }]} onPress={onSubmit}>
              <Text style={styles.buttonText}>Увійти</Text>
            </Pressable>
          </View>

          <Text style={styles.hint}>Демо: будь-яка коректна пошта та пароль від 6
символів</Text>
        </ScrollView>
      </KeyboardAvoidingView>
    </SafeAreaView>
  );
}

// ... оголошення стилів (StyleSheet.create) опущено
```



а) форма входу б) помилка валідації

Рисунок 2.1 - Екран входу

5. Захищена Drawer-навігація

У макеті групи `(drawer)` перевіряється стан авторизації: якщо користувач не увійшов, повертається `<Redirect href="/login" />`, тому жоден екран групи не відкриється навіть за прямим посиланням. Авторизованому користувачу показується `<Drawer>` із власним вмістом меню (`drawerContent`): блок із даними користувача, стандартні пункти навігації (`DrawerItemList`) та кнопка «Вийти», що викликає `logout()`.

Лістинг 2.6 - app/(drawer)/_layout.jsx (без стилів)

```
import { Ionicons } from '@expo/vector-icons';
import { Redirect } from 'expo-router';
import { Drawer, DrawerContentScrollView, DrawerItem, DrawerItemList } from 'expo-
router/drawer';
import { StyleSheet, Text, View } from 'react-native';
import BrandMark from '../../components/BrandMark';
import { colors, fonts, student } from '../../constants/theme';
import { useAuth } from '../../context/AuthContext';


function DrawerContent(props) {
```

```
  const { user, logout } = useAuth();

  return (
    <DrawerContentScrollView {...props} contentContainerStyle={{ flexGrow: 1 }}>
      <View style={styles.profile}>
        <BrandMark size={48} />
        <View style={{ flex: 1 }}>
          <Text style={styles.name} numberOfLines={1}>
            {user?.name}
          </Text>
          <Text style={styles.email} numberOfLines={1}>
            {user?.email}
          </Text>
        </View>
      </View>

      <DrawerItemList {...props} />

      <View style={{ flex: 1 }} />

      <DrawerItem
        label="Вийти"
        onPress={logout}
        icon={({ size }) => <Ionicons name="log-out-outline" size={size}
color={colors.red} />}
        labelStyle={[styles.label, { color: colors.red }]}
      />
      <Text style={styles.footer}>
        ЛР №2 · {student.fullName}, {student.group}
      </Text>
    </DrawerContentScrollView>
  );
}

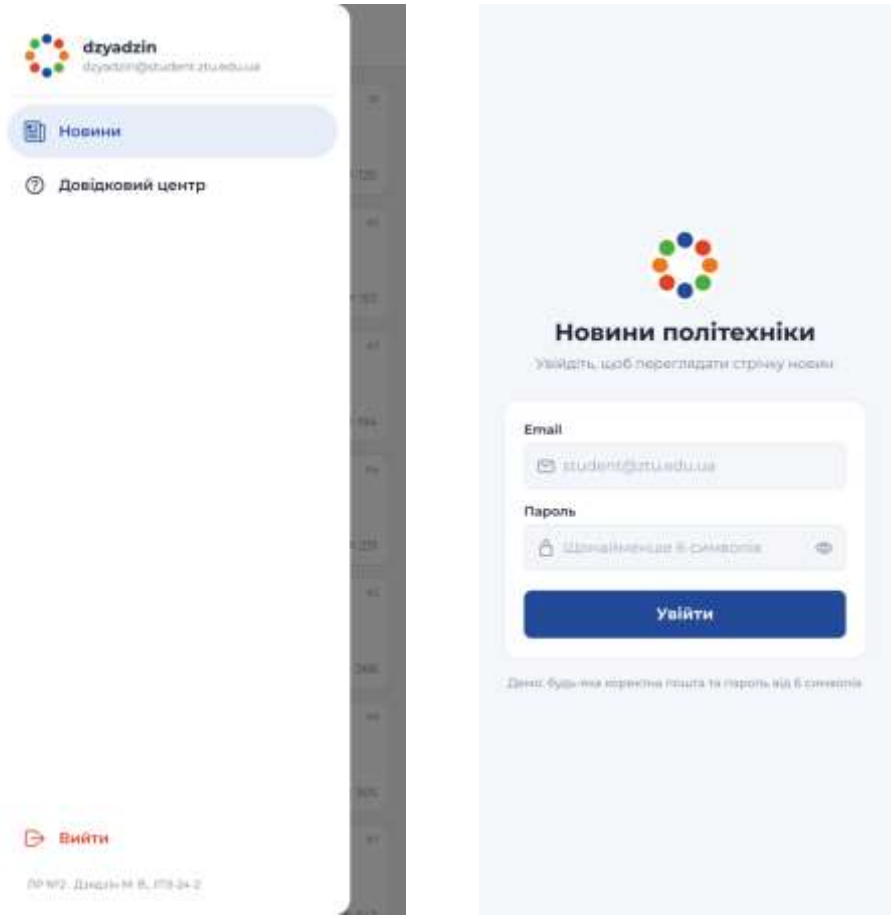
export default function DrawerLayout() {
  const { isAuthenticated } = useAuth();

  // Захищений маршрут: неавторизованого користувача перенаправляємо на вхід
  if (!isAuthenticated) return <Redirect href="/login" />;

  return (
    <Drawer
      drawerContent={(props) => <DrawerContent {...props} />}
      screenOptions={{
        headerTintColor: colors.text,
        headerTitleStyle: { fontFamily: fonts.bold, fontSize: 18 },
        headerStyle: { backgroundColor: colors.surface },
        drawerActiveTintColor: colors.blue,
        drawerActiveBackgroundColor: '#E8EEF9',
        drawerInactiveTintColor: colors.text,
        drawerLabelStyle: styles.label,
        sceneStyle: { backgroundColor: colors.background },
      }}
    >
      <Drawer.Screen
        name="(news)"
        options={{
          title: 'Новини',
          drawerIcon: ({ color, size }) => <Ionicons name="newspaper-outline"
size={size} color={color} />,
        }}
      />
      <Drawer.Screen
        name="faq"
        options={{
          title: 'Довідковий центр',
```

```
          drawerIcon: ({ color, size }) => <Ionicons name="help-circle-outline"
size={size} color={color} />,
        }}
      />
    </Drawer>
  );
}

// ... оголошення стилів (StyleSheet.create) опущено
```



а) бокове меню б) після виходу

Рисунок 2.2 - Бокове меню та вихід із системи

6. Вкладений Stack для розділу новин

Група `(news)` має власний Stack, у якому стрічка новин і екран деталей утворюють стек переходів усередині бокового меню. Заголовок стека вимкнено (`headerShown: false`), щоб він не дублював заголовок Drawer.

Лістинг 2.7 - app/(drawer)/(news)/_layout.jsx

```
import { Stack } from 'expo-router';

// Вкладений Stack усередині Drawer. Заголовок стека вимкнено,
// щоб він не дублював заголовок Drawer.
export default function NewsStackLayout() {
```

```
  return (
    <Stack screenOptions={{ headerShown: false }}>
      <Stack.Screen name="index" />
      <Stack.Screen name="[id]" />
    </Stack>
  );
}
```

7. Стрічка новин на FlatList

Дані генеруються функцією `generateNews()`: на старті 100 новин, кожна порція підвантаження - 20 новин, максимум - 300. У стрічці реалізовано:

- **Pull-to-Refresh** - `RefreshControl` з імітацією запиту до сервера та скиданням стрічки;
- **Infinite Scroll** - `onEndReached` з `onEndReachedThreshold={0.5}`; повторні виклики під час завантаження блокуються через `useRef`, у футері показується індикатор або кількість завантажених новин;
- **getItemLayout** - картки мають фіксовану висоту, тому координати елемента обчислюються без вимірювання розмітки;
- **React.memo** - картка новини мемоїзована, а обробник натискання стабілізовано через `useCallback`, тож уже відображені картки не перерендерюються при додаванні нових;
- параметри віртуалізації `initialNumToRender`, `maxToRenderPerBatch`, `windowSize`, а на Android - `removeClippedSubviews`.

Лістинг 2.8 - app/(drawer)/(news)/index.jsx (без стилів)

```
import { useRouter } from 'expo-router';
import { useCallback, useEffect, useRef, useState } from 'react';
import { ActivityIndicator, FlatList, Platform, RefreshControl, StyleSheet, Text, View
} from 'react-native';
import NewsCard, { ITEM_HEIGHT } from '../../../components/NewsCard';
import { colors, fonts } from '../../../constants/theme';
import { generateNews, INITIAL_COUNT, MAX_COUNT, PAGE_SIZE } from
'../../../data/news';


const LIST_PADDING = 16;


export default function NewsFeedScreen() {
  const router = useRouter();
  const [news, setNews] = useState(() => generateNews(0, INITIAL_COUNT));
  const [refreshing, setRefreshing] = useState(false);
  const [loadingMore, setLoadingMore] = useState(false);
```

```
  const loadingRef = useRef(false);
  const listRef = useRef(null);


  // Лише для web: вимикаємо браузерний scroll anchoring. Інакше після підвантаження
  // Chrome тримає список "приклеєним" до низу й onEndReached спрацьовує знову і
знову.
  // На Android/iOS цей ефект нічого не робить.
  useEffect(() => {
    if (Platform.OS !== 'web') return;
    const node = listRef.current?.getScrollableNode?.();
    if (node?.style) node.style.overflowAnchor = 'none';
  }, []);


  const hasMore = news.length < MAX_COUNT;


  // Pull-to-Refresh: імітуємо запит до сервера та скидаємо стрічку
  const onRefresh = useCallback(() => {
    setRefreshing(true);
    setTimeout(() => {
      setNews(generateNews(0, INITIAL_COUNT));
      setRefreshing(false);
    }, 1200);
  }, []);


  // Infinite Scroll: підвантажуємо наступну сторінку при наближенні до кінця
  const onEndReached = useCallback(() => {
    if (loadingRef.current || !hasMore) return;
    loadingRef.current = true;
    setLoadingMore(true);
    setTimeout(() => {
      setNews((prev) => [...prev, ...generateNews(prev.length, PAGE_SIZE)]);
      setLoadingMore(false);
      loadingRef.current = false;
    }, 900);
  }, [hasMore]);


  // Стабільна функція (useCallback) - інакше React.memo у картці не спрацює
  const openNews = useCallback((id) => router.push(`/${id}`), [router]);


  const renderItem = useCallback(({ item }) => <NewsCard item={item}
onPress={openNews} />, [openNews]);


  // getItemLayout: координати елемента обчислюються без вимірювання розмітки
  const getItemLayout = useCallback(
    (_, index) => ({ length: ITEM_HEIGHT, offset: LIST_PADDING + ITEM_HEIGHT * index,
index }),
    []
  );


  return (
    <FlatList
      ref={listRef}
      data={news}
      keyExtractor={(item) => item.id}
      renderItem={renderItem}
      getItemLayout={getItemLayout}
      contentContainerStyle={styles.list}
      initialNumToRender={8}
      maxToRenderPerBatch={10}
      windowSize={7}
      removeClippedSubviews={Platform.OS === 'android'}
      onEndReached={onEndReached}
      onEndReachedThreshold={0.5}
      refreshControl={
        <RefreshControl refreshing={refreshing} onRefresh={onRefresh}
colors={[colors.blue]} tintColor={colors.blue} />
      }
```

```
      ListFooterComponent={
        <View style={styles.footer}>
          {loadingMore ? (
            <ActivityIndicator color={colors.blue} />
          ) : (
            <Text style={styles.footerText}>
              {hasMore ? `Завантажено ${news.length} новин` : `Це всі новини
(${news.length})`}
            </Text>
          )}
        </View>
      }
    />
  );
}

// ... оголошення стилів (StyleSheet.create) опущено
```

Лістинг 2.9 - components/NewsCard.jsx (без стилів)

```
import { Ionicons } from '@expo/vector-icons';
import { memo } from 'react';
import { Image, Pressable, StyleSheet, Text, View } from 'react-native';
import { colors, fonts, radius } from '../constants/theme';
import { formatDate } from '../data/news';


// Фіксована висота картки потрібна для getItemLayout
export const CARD_HEIGHT = 112;
export const CARD_GAP = 12;
export const ITEM_HEIGHT = CARD_HEIGHT + CARD_GAP;


function NewsCard({ item, onPress }) {
  return (
    <Pressable
      onPress={() => onPress(item.id)}
      style={({ pressed }) => [styles.card, pressed && { opacity: 0.85 }]}
    >
      <Image source={{ uri: item.image }} style={styles.image} />
      <View style={styles.body}>
        <Text style={styles.category}>{item.category.toUpperCase()}</Text>
        <Text style={styles.title} numberOfLines={2}>
          {item.title}
        </Text>
        <View style={styles.meta}>
          <Text style={styles.metaText}>{formatDate(item.date)}</Text>
          <View style={styles.views}>
            <Ionicons name="eye-outline" size={13} color={colors.textMuted} />
            <Text style={styles.metaText}>{item.views}</Text>
          </View>
        </View>
      </View>
      <Text style={styles.number}>#{item.id}</Text>
    </Pressable>
  );
}

// React.memo: вже відрендерені картки не перемальовуються,
// коли батьківський список оновлює свій стан
export default memo(NewsCard);


// ... оголошення стилів (StyleSheet.create) опущено
```



а) стрічка новин б) підвантаження наступної порції

Рисунок 2.3 - Стрічка новин на FlatList

8. Екран деталей новини

Файл `[id].jsx` є динамічним маршрутом. При натисканні на картку викликається `router.push()` з адресою виду `/3`, а на екрані деталей параметр зчитується хуком `useLocalSearchParams()` і за ним формується новина. Оскільки заголовок стека вимкнено, на екран додано власну кнопку «Назад».

Лістинг 2.10 - app/(drawer)/(news)/[id].jsx (без стилів)

```
import { Ionicons } from '@expo/vector-icons';
import { useLocalSearchParams, useRouter } from 'expo-router';
import { Image, Pressable, ScrollView, StyleSheet, Text, useWindowDimensions, View }
from 'react-native';
import { colors, fonts, radius } from '../../../constants/theme';
import { formatDate, makeNewsItem } from '../../../data/news';


export default function NewsDetailsScreen() {
  // Параметр динамічного маршруту /[id]
  const { id } = useLocalSearchParams();
  const router = useRouter();
  const { width } = useWindowDimensions();


  const index = Number(id) - 1;
  const item = Number.isInteger(index) && index >= 0 ? makeNewsItem(index) : null;
```

```
  const goBack = () => (router.canGoBack() ? router.back() : router.replace('/'));

  if (!item) {
    return (
      <View style={styles.center}>
        <Text style={styles.body}>Новину не знайдено</Text>
      </View>
    );
  }

  return (
    <ScrollView contentContainerStyle={{ paddingBottom: 32 }}>
      <Image source={{ uri: item.image }} style={{ width, height: width * 0.6,
backgroundColor: '#DCE3EE' }} />

      <Pressable onPress={goBack} style={styles.back} accessibilityLabel="Назад">
        <Ionicons name="arrow-back" size={20} color={colors.text} />
      </Pressable>

      <View style={styles.content}>
        <View style={styles.badges}>
          <Text style={styles.badge}>{item.category}</Text>
          <Text style={styles.id}>ID: {id}</Text>
        </View>
        <Text style={styles.title}>{item.title}</Text>
        <View style={styles.meta}>
          <Ionicons name="calendar-outline" size={14} color={colors.textMuted} />
          <Text style={styles.metaText}>{formatDate(item.date)}</Text>
          <Ionicons name="eye-outline" size={14} color={colors.textMuted} style={{
marginLeft: 12 }} />
          <Text style={styles.metaText}>{item.views} переглядів</Text>
        </View>
        <Text style={styles.body}>{item.body}</Text>
      </View>
    </ScrollView>
  );
}

// ... оголошення стилів (StyleSheet.create) опущено
```



а) стрічка після підвантаження б) деталі новини /3

Рисунок 2.4 - Підвантаження та екран деталей

9. Довідковий центр на SectionList

Запитання та відповіді згруповано за темами в масив секцій, кожна з яких має заголовок, іконку та масив `data`. Компонент `SectionList` виводить заголовки секцій (липкі, `stickySectionHeadersEnabled`) з лічильником запитань; натискання на запитання розгортає або згортає відповідь.

Лістинг 2.11 - data/faq.js (фрагмент)

```
export const faqSections = [
  {
    title: 'Обліковий запис',
    icon: 'person-circle-outline',
    data: [
      { id: 'a1', q: 'Як увійти в застосунок?', a: 'Введіть корпоративну пошту та пароль на екрані входу. Після успішної авторизації вас буде автоматично перенаправлено до стрічки новин.' },
      { id: 'a2', q: 'Як вийти з облікового запису?', a: 'Відкрийте бокове меню та натисніть кнопку «Вийти» внизу. Ви повернетеся на екран входу.' },
      { id: 'a3', q: 'Я забув пароль. Що робити?', a: 'Зверніться до служби підтримки університету або скористайтеся формою відновлення пароля на сайті.' },
    ],
  },
  // ... інші секції: «Стрічка новин», «Сповіщення», «Технічні питання»
];
```

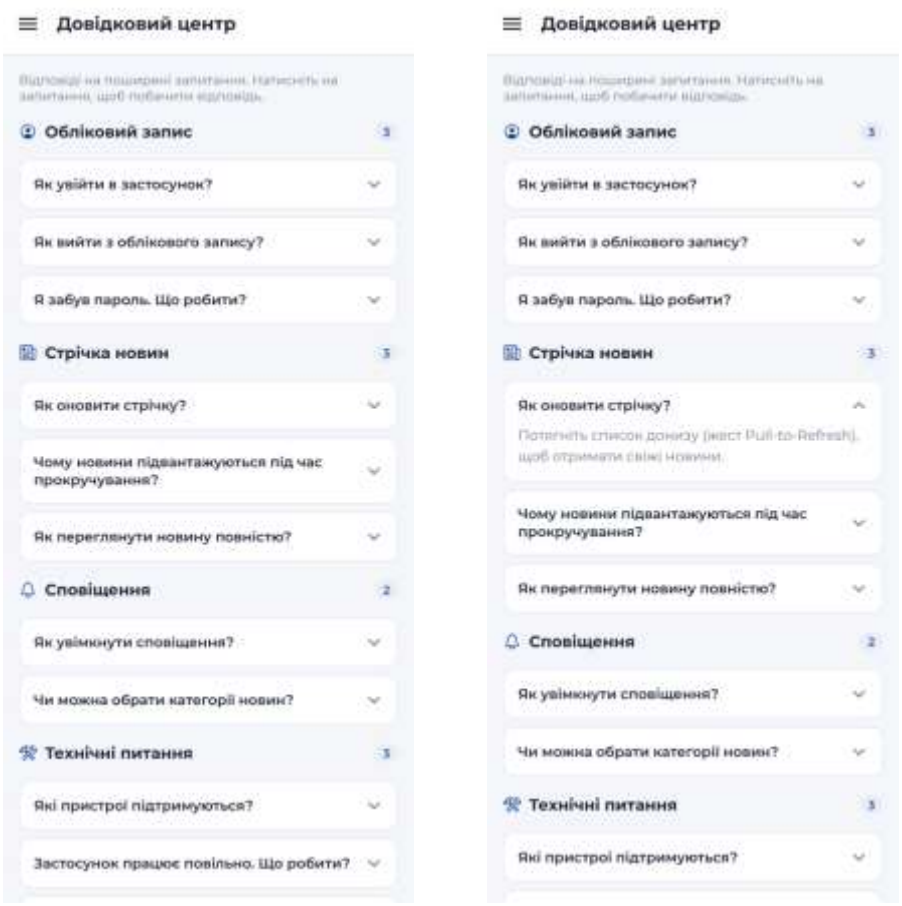
Лістинг 2.12 - app/(drawer)/faq.jsx (без стилів)

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { Pressable, SectionList, StyleSheet, Text, View } from 'react-native';
import { colors, fonts, radius } from '../../constants/theme';
import { faqSections } from '../../data/faq';


export default function FaqScreen() {
  const [openId, setOpenId] = useState(null);


  return (
    <SectionList
      sections={faqSections}
      keyExtractor={(item) => item.id}
      stickySectionHeadersEnabled
      contentContainerStyle={styles.list}
      extraData={openId}
      ListHeaderComponent={
        <Text style={styles.intro}>Відповіді на поширені запитання. Натисніть на
запитання, щоб побачити відповідь.</Text>
      }
      renderSectionHeader={({ section }) => (
        <View style={styles.sectionHeader}>
          <Ionicons name={section.icon} size={18} color={colors.blue} />
          <Text style={styles.sectionTitle}>{section.title}</Text>
          <Text style={styles.sectionCount}>{section.data.length}</Text>
        </View>
      )}
      renderItem={({ item }) => {
        const open = openId === item.id;
        return (
          <Pressable style={styles.item} onPress={() => setOpenId(open ? null :
item.id)}>
            <View style={styles.questionRow}>
              <Text style={styles.question}>{item.q}</Text>
              <Ionicons name={open ? 'chevron-up' : 'chevron-down'} size={18}
color={colors.textMuted} />
            </View>
            {open && <Text style={styles.answer}>{item.a}</Text>}
          </Pressable>
        );
      }}
      ItemSeparatorComponent={() => <View style={{ height: 8 }} />}
      SectionSeparatorComponent={() => <View style={{ height: 8 }} />}
    />
  );
}


// ... оголошення стилів (StyleSheet.create) опущено
```



а) список запитань б) розгорнута відповідь

Рисунок 2.5 - Довідковий центр

10. Запуск, тестування та розміщення на GitHub

Застосунок запускався командою `npx expo start` на смартфоні через Expo Go та в емуляторі Android. Перевірено: перенаправлення на вхід при відкритті захищених маршрутів без авторизації, валідацію форми входу, вхід і автоматичний перехід до стрічки, підвантаження порцій до 300 новин, оновлення жестом, відкриття деталей і повернення назад, роботу довідкового центру та вихід із системи.

Вихідний код розміщено у репозиторії https://github.com/ipz242dmv-alt/MobileLabsRN2026 у папці lab2 разом із README.md.

ВІДПОВІДІ НА КОНТРОЛЬНІ ЗАПИТАННЯ

1. Як працює вкладена навігація в архітектурі Expo Router?

У Expo Router навігатори вкладаються через структуру папок. Кожна папка з файлом `_layout.jsx` - окремий навігатор, а екрани в ній є його маршрутами. У цьому проєкті кореневий `app/_layout.jsx` - це Stack, усередині нього `(drawer)/_layout.jsx` - Drawer, а в ньому `(news)/_layout.jsx` - ще один Stack. Папки в дужках є групами маршрутів: вони впливають на вкладеність навігаторів, але не додаються до URL. Перехід між екранами вкладеного Stack додає екран у стек усередині Drawer, а меню й заголовок Drawer лишаються на місці.

2. Яким чином реалізується захист маршрутів від неавторизованого доступу?

Стан авторизації зберігається в React Context, провайдер якого обгортає всю навігацію в кореневому макеті. Макет захищеної групи `(drawer)/_layout.jsx` зчитує `isAuthenticated` і, якщо користувач не увійшов, повертає `<Redirect href="/login" />` замість навігатора, тому жоден екран групи не рендериться без входу. Екран входу працює навпаки: авторизованого користувача перенаправляє на `/`. Після `logout()` стан змінюється, макет перерендерюється, і користувача повертає на вхід. Альтернативно можна використати компонент `<Stack.Protected guard={...}>`.

3. Чим FlatList відрізняється від ScrollView та що таке віртуалізація списків?

`ScrollView` рендерить одразу всі дочірні елементи, навіть невидимі, що для сотень карток означає велике споживання пам’яті та повільний перший рендер. `FlatList` побудований на `VirtualizedList` і рендерить лише елементи у видимій області та невеликому буфері навколо неї; елементи, що вийшли за межі вікна, демонтуються, а на їхньому місці лишається порожній простір потрібної висоти. Це і є віртуалізація: розмір дерева компонентів залежить від розміру екрана, а не від кількості даних. Крім того, `FlatList` має вбудовані Pull-to-Refresh, `onEndReached`, роздільники, заголовок і футер.

4. Як отримати параметри динамічного маршруту на цільовому екрані в Expo Router?

Файл із назвою у квадратних дужках, наприклад `[id].jsx`, є динамічним маршрутом. На цільовому екрані параметр отримують хуком `useLocalSearchParams()`: `const { id } = useLocalSearchParams();`. Параметри надходять рядками, тому за потреби їх перетворюють (`Number(id)`). Передати параметр можна шляхом (`router.push('/5')`, `<Link href="/5">`) або об'єктом (`router.push({ pathname: '/[id]', params: { id: 5 } })`).

5. Яку структуру даних приймає компонент SectionList?

Властивість `sections` - це масив об'єктів-секцій. Кожна секція має обов'язкове поле `data` (масив елементів секції) та довільні додаткові поля, наприклад `title` чи `icon`, які використовуються в `renderSectionHeader`. Функція `renderItem` отримує `{ item, index, section }`, а `renderSectionHeader` - `{ section }`.

ВИСНОВКИ

У ході лабораторної роботи реалізовано ієрархічну навігацію Expo Router (Stack → Drawer → Stack) з групами маршрутів, глобальний стан авторизації через React Context та захист маршрутів за допомогою `<Redirect>`. Стрічку із сотнями новин побудовано на `FlatList` з оновленням жестом, нескінченним прокручуванням і оптимізаціями `getItemLayout`, `React.memo` та стабільними колбеками. Довідковий центр зі згрупованими даними реалізовано на `SectionList`, а екран деталей отримує параметр динамічного маршруту через `useLocalSearchParams`. Віртуалізовані списки забезпечують плавне прокручування незалежно від обсягу даних, а файлова маршрутизація робить структуру навігації наочною.